Alumno/a:	Arturo Hernanz Carrasco-Muñoz	NIA:	100495850



Memoria Proyecto Final

Titulación: GRADO INGENIERÍA INFORMÁTICA Año Académico: 2024/25 -- Curso: 3º Asignatura: Sistemas Distribuidos	uc3m | Universidad Carlos III de Madrid

Índice:

1. **Aclaración inicial**
2. **Arquitectura desarrollada**
3. **Estructura de los mensajes**
4. **Protocolos de aplicaciones**
5. **Justificación del paradigma seleccionado**
6. **Operaciones de cada componente**
7. **Pruebas realizadas**
8. **Conclusión**

1. Aclaración inicial:

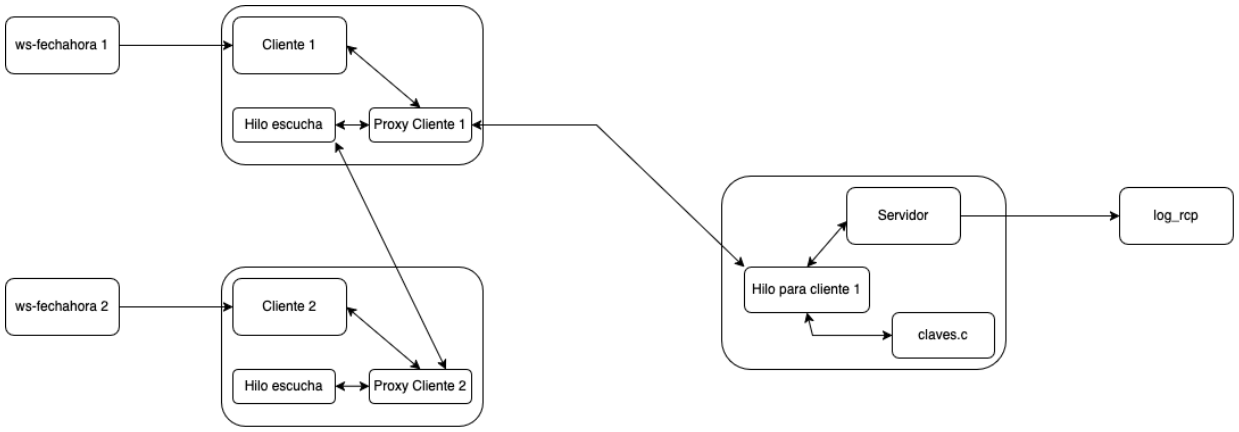
Antes de explicar el funcionamiento del sistema desarrollado debo aclarar que malinterpreté el enunciado de la siguiente manera:

Cuando en el enunciado se pide desarrollar un cliente en lenguaje "***Python***" y el servidor en lenguaje "***C***" yo entendí que podía desarrollar un cliente en "***Python***", encargado de recibir los comandos del usuario, el servidor en "***C***" y entre ambos, siguiendo el mismo estilo que los ejercicios evaluables, un proxy que se encargara de hacer las llamadas al servidor, también en lenguaje "***C***".

Más adelante aclararé esta arquitectura.

Claramente este fue un error mío que me ha llevado a desarrollar un sistema más complejo de lo que se pedía.

2. Arquitectura desarrollada:



Para el desarrollo de este sistema se han desarrollado 5 componentes principales:

- ***client.py*:** Parte del sistema con la que interactúa el usuario, en él ejecuta las diferentes llamadas a las funciones disponibles, permitiendo tanto una comunicación ***cliente - servidor*** como una comunicación ***peer to peer***.

- ***servidor.c*:** Parte del sistema que recibe las peticiones de los clientes y ejecuta las funciones sobre la base de datos. Es el único que tiene acceso a la base de datos.

- ***proxy.c*:** Servicio intermedio entre el cliente y el servidor y entre los diferentes clientes, a partir de este se ha desarrollado una librería dinámica (***libclaves.so***).
  Este sistema le permite al cliente ejecutar las diferentes funciones, comunicándose con el servidor o los demás clientes y devolviendo el resultado al cliente.

- ***ws-fechahora-service.py*:** Servicio web al que llama el cliente para devolver la fecha y hora de llamada a una función.

- ***log_rpc.c*:** Servicio llamado por el servidor para imprimir el nombre, la función y la fecha cuando un usuario llama a una función.

Para permitir la concurrencia, el servidor crea un hilo por cada cliente que se le conecta, del mismo modo, el proxy del cliente crea un hilo por cada petición de los demás clientes.

Para permitir el buen funcionamiento de este sistema se hace uso de los siguientes ficheros secundarios o de apoyo:

- ***claves.c/claves.h*:** Fichero donde se han desarrollado las diferentes funciones que se ejecutarán sobre la base de datos, tanto funciones principales como secundarias/auxiliares.

- ***estructuras.c*:** Fichero donde se han definido las diferentes estructuras que se usan tanto en la base de datos como en las comunicaciones.

- ***lines.c/lines.h:*** Fichero proporcionado en "***Aula global***" donde se definen las funciones usadas para la comunicación por "***sockets"***.

- **Ficheros rpc:** Para el funcionamiento del servicio rcp se ha desarrollado un fichero ***"tuplas.x"*** a partir del cual se generan, a través del comando "***rpcgen -CNM tuplas.x***" (**C:** Flag de "***C***" moderno, **N:** Flag de varios argumentos por función, **M:** Flag para conexión multihilo) los stubs del cliente que hace de cliente y del que hace de servidor.
  Se han modificado ligeramente algunos de los ficheros generados automáticamente para eliminar "***warnings"*** e incluir funciones no generadas automáticamente.

Por último, en cuanto a la comunicación ***"peer to peer"*** entre los clientes, cada cliente crea un hilo de escucha de peticiones al registrarse.
Este hilo crea a su vez un hilo por cada petición, de esta forma se permiten llamadas concurrentes.
Cabe destacar que, al contrario que con los hilos del servidor, estos solo bloquean la copia de la petición, ya que, no hay posibilidad de carrera de datos al ser operaciones de solo lectura.

3. Estructura de los mensajes:

Tanto la comunicación "***cliente-servidor***" como "***peer to peer***" se comunican a través de sockets haciendo uso de las funciones desarrolladas en los ficheros "***lines.c/lines.h***".

Sin embargo, hay que destacar tres aspectos de las comunicaciones comentadas previamente:

- Por un lado, toda la información a transmitir entre el cliente que realiza una operación y el servidor u otro cliente (***peer to peer***) se agrupa previamente en estructuras de tipo "***Peticion***", para más tarde enviar cada campo individualmente a través de los sockets.
  Después de transmitirla se vuelve a agrupar en este tipo de estructuras para una gestión más cómoda.

- Por otro lado, en cuanto a las funciones "***list_user***" y "***list_content***" que envían una lista de información (usuarios conectados y contenido publicado, respectivamente), para ello envían primero la cantidad elementos de la lista y luego cada elemento, mientras, el cliente recibe una por una y las muestra por pantalla de forma secuencial.

- Por último, en cuanto a la transmisión de ficheros entre usuarios se envía primero el tamaño máximo del fichero y, más tarde, paquetes de hasta 1024 bytes hasta enviar todo el fichero.

4. Protocolos de aplicaciones:

A continuación se detalla el protocolo de cada aplicación principal:

- ***cliente.py:*** Cada vez que se introduce un comando válido, se llama a la función correspondiente que llama a las funciones del fichero "***proxy.c***" pasándole antes cada argumento codificándolo para que esta segunda función pueda usarlo.

- ***proxy.c:*** Este fichero ofrece las funciones de comunicación entre el cliente y el servidor u otros clientes.
  El fichero se divide, generalmente, en dos partes:
  - Comunicación ***cliente-servidor*****:** Esta parte del código permite la comunicación entre el cliente y el servidor a través de sockets, el cliente (mediante el proxy) envía las peticiones y espera la respuesta del servidor para, más tarde, darle esta respuesta al cliente.
  - Comunicación ***peer to peer:*** El funcionamiento es el mismo que el apartado anterior adaptándolo a la particularidad de la comunicación entre clientes.
    Uno le envía peticiones a otro y espera la respuesta o, por otro lado, espera las peticiones de otros clientes y les envía la respuesta.

  Para que la comunicación entre el fichero "***Python***" del cliente y el "***C***" del proxy pueda llevarse a cabo se ha enlazado la librería mediante "***ctypes***", que especifica el tipo de dato que recibe como parámetros además de los tipo de valores que devuelven cada función.

- ***ws-fechahora-service.py:*** Este fichero es el que ejecuta el servicio web, este se lanza en la misma ip que el cliente que lo lanza a través de "***localhost***" y ofrece una única función que devuelve la fecha y hora actual en el momento de la llamada a la función.
  Las llamadas a este servicio por parte del cliente se llevan a cabo para conocer el momento exacto en el que se ejecutan las funciones para poder transmitir esta información al servidor.

- ***servidor.c:*** Este programa es el que gestiona toda la información de la base de datos, cuando recibe una petición del cliente la ejecuta mediante las funciones que ofrece el fichero "***claves.c***" y devuelve el resultado al cliente.

  Por otro lado, en el contexto de la funcionalidad "***cliente - servidor rpc***" este servidor actúa como cliente de ***"log_rpc.c"*** que a su vez actúa de servidor.
  Le envía el nombre del usuario que hace la petición, la función que ha ejecutado y la fecha y hora para que este otro servidor pueda registrarlo.

- ***log_rpc.c:*** Este fichero actúa como servidor en el contexto mencionado previamente, por lo que ofrece una función que el cliente puede invocar de forma remota (***RPC***) y obtener su respuesta.
  Esta funcionalidad solamente registra mediante un log en pantalla quién hace cada operación y cuando lo hace.

5. Justificación del paradigma seleccionado:

A continuación se justificarán las decisiones de diseño más relevantes.

En primer lugar, y dejando de lado que haya sido una equivocación por malinterpretar el código, el uso de un paso intermedio en la comunicación entre el cliente y el servidor o entre clientes tiene algunas ventajas, como el uso de un lenguaje que permita un mayor control sobre las funcionalidades que se llevan a cabo y sobre los datos con los que se trata, también permite una mejor organización del código, pues, hasta cierto punto separar los ficheros por funcionalidad (interacción con el usuario por un lado e interacción con el resto de partes de sistema por otro) ayuda a aclarar las diferentes partes, por último, evita que el usuario final acceda a partes sensibles del código que no está destinado a manejar, evitando posibles fallos o situaciones que no permitan el normal desempeño de las funcionalidades del sistema.

Por otro lado, la elección de una serie de estructuras estándares para la agrupar los datos permite estandarizar el comportamiento del código, facilitando las funciones de desarrollo y mantenimiento del mismo.

En cuanto a la base de datos, se ha decidido hacer uso de una lista doblemente enlazada, que permite navegar en ambos sentidos (hacia el final y hacia el principio de la lista) con una complejidad de inserción de O(1), además, para no aumentar la complejidad de búsqueda de O(n) se han desarrollado funciones auxiliares que permiten extraer aquellas estructuras (***nodos o publicaciones***) haciendo una única búsqueda del elemento, de esta forma no se recorre la lista varias veces en busca de un mismo elemento.

En cuanto a los elementos de la base de datos, se ha hecho un amplio uso de punteros, lo que permite trabajar con datos de tamaños variables (dentro de unos estándares fijados por el enunciado) reduciendo el uso de la memoria.

6. Operaciones de cada componente:

En este apartado se detallarán las principales funciones de los componentes principales:

client.py: Este fichero cuenta con una función por cada funcionalidad disponible en el sistema: registrar un usuario, eliminarlo de la base de datos, conectarlo, desconectarlo, publicar un fichero, eliminarlo, obtener una lista de los usuarios conectados, una de los ficheros publicados por cada usuario y la función para descargar un fichero publicado por otro usuario.

Cada una de estas funciones llama a la correspondiente función de "***proxy.c***", habiendo convertido los datos precisos que requieren como argumentos.

proxy.c: Las principales funciones de este fichero se pueden dividir en las siguientes categorías:

- Comunicación con el servidor: Esta sección está compuesta únicamente por la función "***comunicacion_servidor***", que recibe una petición y envía sus campos al servidor, y dependiendo de qué operación se trataba, espera la respuesta de una manera u otra (respuesta simple o varias).

- Comunicación con otros clientes: Esta sección está compuesta por las funciones "***recibir_cliente***", esta función es la que se ejecuta en cada hilo creado por el hilo de escucha, ejecuta la parte emisora de la función "***get_file***", es decir, transmite el contenido del fichero a otros clientes, "***comunicacion_cliente***", función encargada de solicitar y recibir el fichero por parte de otros clientes y "***scuchar_socket_cliente***", función que se ejecuta en el hilo de escucha, encargada de crear un hilo por cada solicitud.

- Funciones principales: Esta sección comprende a las funciones "***enviar_peticion***" y "***recibir_peticion***", encargadas de enviar y recibir los campos de las estructuras de tipo "***Peticion***"

- Envío y recibo de peticiones: Aquí se agrupan las funciones principales, aquellas a las que se llama desde el fichero "***client.py***" y que se encargan de completar los campos de las estructuras de tipo "***Peticion***" para pasarlas a las funciones "***comunicacion_servidor***" o "***comunicacion_cliente***" en su caso.

servidor.c: Este fichero contiene tres funciones:

- ***main:*** Encargada de almacenar los datos de los sockets del servidor y de cada cliente, además de crear un hilo por cada petición recibida de cada cliente.

- ***recibir_socket:*** La función que se ejecuta en cada hilo, envía los datos de log al servidor rpc y llama a la función correspondiente del fichero "***claves.c***" y envía la respuesta al cliente.

  En caso de que el cliente ejecute las funciones de listado, esta función envía al cliente cada elemento de la lista en su formato correspondiente.

- ***recibir_peticion:*** Esta función recibe cada campo de la estructura de tipo "***Peticion***" y completa los campos de una estructura del mismo tipo de carácter local.

claves.c: Este fichero contiene la ejecución de cada función principal sobre la base de datos ("***register_user***", "***unregister***", "***connect_user***", "***disconnect***", "***publish***" y "***delete***"), así como, de las auxiliares que sirven para comprobar la existencia de un nodo, publicación o el estado de conexión de un cliente y devuelven un código de error o el nodo, publicación o datos de conexión correspondientes.

ws-fechahora-service.py: La principal función de este componente es ***"fechahora",*** que devuelve la fecha y hora en el momento de llamarla.

log_rpc.c: Este fichero cuenta con dos funciones: "***log_1_svc***", que imprime por pantalla el nombre de usuario que ha realizado la función, qué función ha realizado y la hora, y por otro lado "***log_filename_1_svc***", que además de todo lo que imprime la anterior función, esta imprime el nombre del fichero (esta función se llama cuando el cliente ha hecho uso de las funciones "***publish***" o "***delete***")

7. Pruebas realizadas:

Para probar el sistema desarrollado, además de ejecutar cada funcionalidad individualmente y colectivamente en "***Guernika***" se han desarrollado dos ficheros de pruebas:

- ***test_cliente.py:*** En este fichero se comprueba el buen funcionamiento de las funcionalidades para un cliente, es decir, todas menos "***get_file***".
- ***test_error.py:*** En este fichero se comprueba la correcta detección de errores en cada funcionalidad.

Estos ficheros prueban cada función presente en el fichero ***"client.py"*** en lugar de probar el propio fichero, ya que, requiere una interacción entre el usuario y el programa.
Para ejecutar los fichero hay que ejecutar por un lado:

```
None
./servidor_log &
env LOG_RCP_IP=localhost ./servidor -p 5850
```

Y por otro:

```
None
./nombre_del_fichero_de_test
```

Es importante destacar que se ha probado sobre el puerto 5850 del servidor y sobre la IP de "***guernika"*** en el momento de la prueba, en caso de cambiar habría que modificar las primeras líneas de estos ficheros que actualmente están de la siguiente manera:

```
None
client._server = "163.117.142.41"
client._port = 5850
```

He de destacar que no he conseguido incluir la función "***get_file"*** en los ficheros de prueba debido a la necesidad de ejecutarlo en dos clientes al mismo tiempo, sin embargo, a la hora de probarlo haciendo uso del fichero ***"client.py"*** se comprobó su correcto funcionamiento.
Para poder probarla, hubo que ejecutar un cliente en "***guernika***" y otro en "***guernika-gpu***", ya que, no compartían IP.
Este mismo comportamiento es el que no supe replicar en los ficheros de prueba.

8. Conclusión:

Para cerrar la presente memoria, he de destacar el interés que me ha despertado este trabajo, siendo uno de los más interesantes a la par de entretenido que he realizado y de los que más he aprendido, ya que, hacerlo solo me ha forzado a planificarme mejor y me ha hecho dedicarle esfuerzo a todas las partes de la práctica por igual.

Es posible que por haber malinterpretado el enunciado haya complicado más de la cuenta el sistema o haya podido perder un poco de tiempo, pero ahora solo puedo ver los aspectos positivos.

El único aspecto negativo que destacaría es el uso de "***RPC***", que me obliga a depender de un código automático que, además, he tenido que corregir y completar que, sin embargo, puede llegar a ser útil en algunos aspectos.